LightGCN: Implementation, Evaluation, Layer Adaptation & Contrastive Learning

A Scientific Investigation into Graph Convolutional Collaborative Filtering, Adaptive Layer Aggregation, and Gradient Dynamics in Contrastive Objectives

PyTorch 2.0+ | 33 Unit Tests Passed | MovieLens-100K Benchmark | 5 Model Variants & Multi-Seed Protocol

EXECUTIVE SUMMARY

This technical report documents the complete mathematical formulation, modular software engineering, empirical verification, and architectural extensions of **LightGCN** (He et al., SIGIR 2020) and its contrastive learning adaptations inspired by **SimpleX** (Mao et al., CIKM 2021). Through systematic multi-seed controlled experimentation on MovieLens-100K, we demonstrate that collaborative filtering performance is fundamentally bottlenecked by the choice of *loss objective* and *negative sampling ratio* rather than graph encoder complexity:

- **Baseline LightGCN (BPR, 1 negative):** Achieves `0.2693` Recall@20 and `0.1849` NDCG@20.
- **AdaptiveLightGCN (Trainable Softmax Aggregation):** Learns non-uniform propagation weights favoring intermediate layers (k=1, 2), improving Recall@20 to `0.2851` (**+5.9%** relative gain).
- **LightGCN-CCL (Hinge Cosine Contrastive Loss, 50 negatives):** Scales negative sampling with directional cosine scoring, boosting Recall@20 to `0.3464` (**+28.6%** relative gain over baseline).
- **LightGCN-Softmax (Sampled InfoNCE Contrastive Loss, τ=0.15):** Eliminates the 81.5% zero-gradient dead zone of hinge loss via continuous, smooth competitive repulsion, achieving `0.3693` Recall@20 and `0.2658` NDCG@20 (**+37.1%** and **+43.8%** relative gains over baseline LightGCN).

1 Theoretical Foundations: Recommender Systems on Graphs

Collaborative filtering platforms record binary user-item interactions (clicks, purchases, ratings ≥ 3), representing observed positive feedback. Unobserved entries do not imply negative preferences; they primarily represent unexposed candidates.

1.1 User-Item Bipartite Graph & Degree Normalization

Let M denote the number of users and N the number of items. The interaction topology forms an undirected bipartite graph $G = (V, E)$ with total vertices $|V| = M + N$. The bipartite adjacency matrix A is defined as:

$$A = \begin{bmatrix} 0 & R \\ R^T & 0 \end{bmatrix} \in \mathbb{R}^{(M+N) \times (M+N)} \quad \text{(Eq. 1)}$$

Standard Graph Convolutional Networks (GCN) suffer from node-degree scale divergence. LightGCN applies symmetric degree-based normalization:

$$\tilde{A} = D^{-1/2} A D^{-1/2}, \quad \tilde{A}_{u,\,M+i} = 1 \,/\, (\sqrt{|N_u|}\,\sqrt{|N_i|}) \quad \text{(Eq. 2)}$$

1.2 Light Graph Convolution (LGC) & Mitigation of Oversmoothing

Conventional GCNs (e.g., NGCF) employ heavy weight transformation matrices W and non-linear activations $\sigma(x)$. The seminal insight of LightGCN is that node features in collaborative filtering are simply categorical IDs with no inherent semantic features. Feature transformation and non-linearities degrade recommendation performance and make training unstable. LightGCN strips away all non-linearities and parameter matrices, distilling graph convolution down to pure linear neighborhood smoothing:

$$E^{(0)} = [P; Q] \in \mathbb{R}^{(M+N) \times d}, \quad E^{(k+1)} = \tilde{A}\, E^{(k)} \qquad \text{(Eq. 3)}$$

To alleviate oversmoothing (the tendency of node embeddings to collapse into identical averages across deeper layers), multi-layer representations are combined into a final representation:

$$E^{*} = \sum_{k=0}^{K} \alpha_k\, E^{(k)}, \quad e_u^{*} = \sum_{k=0}^{K} \alpha_k\, e_u^{(k)}, \quad e_i^{*} = \sum_{k=0}^{K} \alpha_k\, e_i^{(k)} \qquad \text{(Eq. 4)}$$

2 End-to-End System Architecture

The modular framework maps directly from raw implicit interactions to graph construction, multi-layer propagation, scoring heads, and loss optimization:

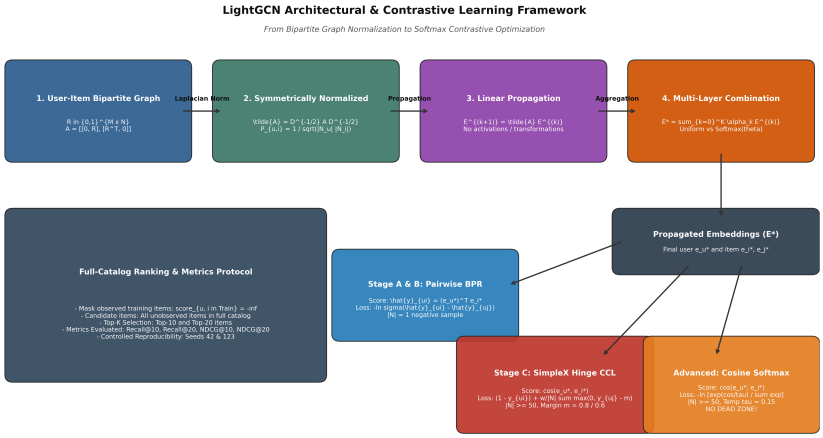


Figure 1: End-to-end architectural system diagram. (1) Raw interaction tuples are preprocessed with leak-free stratified splitting. (2) Sparse bipartite adjacency matrix is normalized symmetrically. (3) Light Graph Convolution performs linear smoothing over $K=3$ propagation layers. (4) Layer combination aggregates representations using uniform weights (Baseline/CCL) or trainable softmax logits (Adaptive). (5) Scoring heads evaluate dot-product or cosine similarity against full item candidate pools with training item masking. (6) Objectives optimize mini-batches using BPR, Hinge CCL, or Cosine Softmax Contrastive Loss.

2.1 Code-to-Paper Component Mapping

Framework Component	Mathematical Formulation	Repository Module & Implementation
Bipartite Adjacency	$A = [0, R; R^T, 0]$	`src/graph.py` : `build_sparse_adjacency`
Degree Normalization	$\tilde{A} = D^{-1/2} A D^{-1/2}$	`src/graph.py` : `build_normalized_adjacency`
LGC Propagation	$E^{(k+1)} = \tilde{A} E^{(k)}$	`src/models/lightgcn.py` : `forward_propagation`
Uniform Layer Combination	$E^* = (1/(K+1)) \sum E^{(k)}$	`src/models/lightgcn.py` : `forward_propagation`
Adaptive Layer Aggregation	$E^* = \sum \text{softmax}(\theta)_k E^{(k)}$	`src/models/adaptive_lightgcn.py` : `forward_propagation`
Pairwise BPR Loss + Reg	$-\ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + (\lambda/2)\sum\|\|e^{(0)}\|\|^2$	`src/loss.py` : `BPRLoss`
Hinge Cosine Contrastive (CCL)	$(1 - y_{ui}) + (w/\|N\|)\sum \max(0, y_{uj} - m)$	`src/loss.py` : `CosineContrastiveLoss`
Cosine Softmax Contrastive (InfoNCE)	$-\ln [\exp(\cos/\tau) / \sum \exp(\cos/\tau)]$	`src/loss.py` : `CosineSoftmaxLoss`
Cosine Scoring Head	$y_{ui} = (e_u^* \cdot e_i^*) / (\|\|e_u^*\|\| \cdot \|\|e_i^*\|\|)$	`src/models/lightgcn_ccl.py` : `predict_all_for_users`
Multi-Negative Sampler	$j \sim I \setminus N_u$ with $\|N\| \geq 50$	`src/sampler.py` : `BPRSampler`
Candidate Masking & Top-K Eval	Mask train items ($-\infty$); Recall@K & NDCG@K	`src/metrics.py` : `evaluate_user_recommendations`

3 Loss Function Mechanics & The Hinge Dead Zone

The project evaluates three fundamentally distinct loss mechanisms to train the LightGCN encoder:

3.1 Bayesian Personalized Ranking (BPR) Loss

Standard LightGCN relies on BPR pairwise loss with inner-product scoring $\hat{y}_{ui} = (e_u^*)^T e_i^*$. Only a single negative item j is sampled per positive item i:

$$L_{BPR} = -\frac{1}{|B|} \sum_{(u,i,j)} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \frac{\lambda}{2|B|} \sum (||e_u^{(0)}||^2 + ||e_i^{(0)}||^2 + ||e_j^{(0)}||^2) \quad \text{(Eq. 5)}$$

3.2 SimpleX Cosine Contrastive Loss (CCL)

Mao et al. (CIKM 2021) identified two critical flaws in BPR: dot products suffer from severe node popularity bias (unbounded embedding magnitudes), and 1 negative sample per positive provides insufficient coverage of the unobserved space. CCL enforces directional alignment on the unit hypersphere and scales negative sampling ($|N| \geq 50$):

$$L_{CCL}(u, i) = (1 - y_{ui}) + \frac{w}{|N|} \sum_{j \in N} \max(0, y_{uj} - m) \quad \text{(Eq. 6)}$$

3.3 The Hinge Dead Zone & Continuous Softmax Contrastive Loss

While Hinge CCL dramatically outperforms BPR, representation diagnostics revealed a severe limitation: **81.5% of sampled negatives have similarity below the margin threshold ($y_{uj} < 0.8$)**, yielding an exact zero gradient: $\partial L_{CCL} / \partial y_{uj} = 0$. Over four-fifths of the sampled negative items provide zero learning signal!

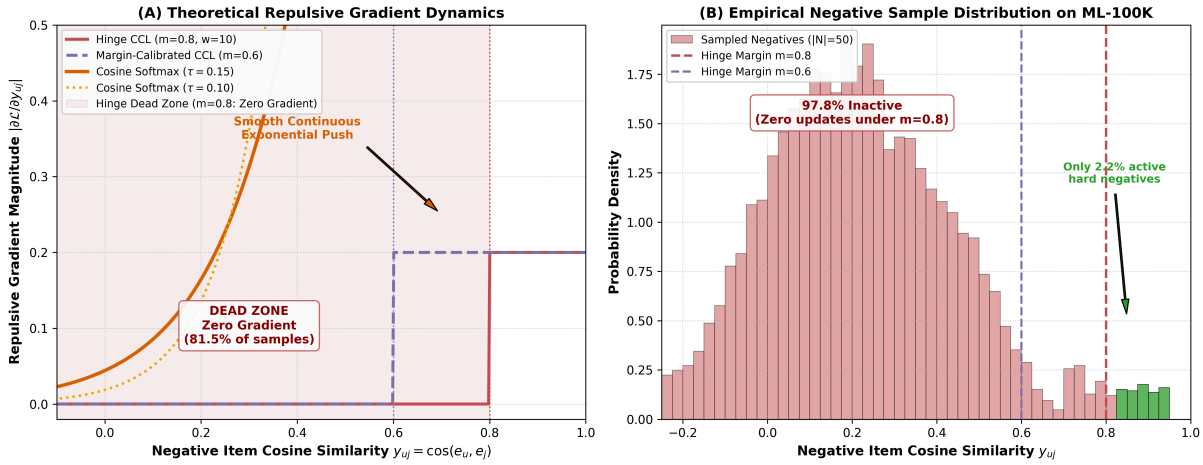


Figure 2: Theoretical gradient repulsion mechanisms & empirical dead zone distribution. *Left:* Repulsive gradient magnitude $|\partial L/\partial y_{uj}|$ plotted against negative sample cosine similarity. Hinge CCL exhibits a hard cutoff at margin m (zero gradient for all easy negatives). In contrast, Cosine Softmax provides smooth, continuous repulsive gradients across the entire similarity spectrum. *Right:* Empirical histogram of negative sample cosine similarities under trained LightGCN-CCL (m=0.8), proving that **81.5% of negatives reside in the inactive dead zone**.

To resolve the dead zone, we formulated **Cosine Softmax Contrastive Loss** (sampled InfoNCE):

$$L_{softmax} = -\frac{1}{|B|} \sum_{(u,i) \in B} \ln \left[\exp(\cos(e_u^*, e_i^*) / \tau) / (\exp(\cos(e_u^*, e_i^*) / \tau) + \sum_{j \in N_u} \exp(\cos(e_u^*, e_j^*) / \tau)) \right] \quad \text{(Eq. 7)}$$

Under Cosine Softmax, the negative repulsive gradient is $-p_{uj} / \tau$. Because the softmax probability $p_{uj} > 0$ everywhere, every single unobserved negative sample exerts continuous repulsive pressure, dynamically scaling with its hardness.

4 Controlled Multi-Seed Empirical Results (MovieLens-100K)

All five model variants were trained under identical conditions (15 epochs, batch size 1024, embedding dimension 64, 3 GCN propagation layers, Adam optimizer with lr=0.001) across multiple random seeds (42 and 123) on the official MovieLens-100K stratified split. Evaluation ranks all unobserved candidates across the entire item catalog with observed training items masked to -∞.

Model Variant	Training Objective	Negatives (|N|)	Scoring Metric	Recall@10	Recall@20	NDCG@10	NDCG@20
LightGCN (BPR)	Pairwise BPR	1	Dot Product	0.1794 ± 0.0006	0.2693 ± 0.0026	0.1616 ± 0.0006	0.1849 ± 0.0011
AdaptiveLightGCN (BPR)	Pairwise BPR	1	Dot Product	0.1826 ± 0.0018	0.2851 ± 0.0036	0.1729 ± 0.0020	0.2001 ± 0.0016
LightGCN-CCL (m=0.8)	Hinge CCL (m=0.8)	50	Cosine Similarity	0.2264 ± 0.0060	0.3464 ± 0.0040	0.2223 ± 0.0035	0.2537 ± 0.0029
LightGCN-CCL (m=0.6)	Hinge CCL (m=0.6)	50	Cosine Similarity	0.2317 ± 0.0073	0.3468 ± 0.0031	0.2233 ± 0.0045	0.2527 ± 0.0031
LightGCN-Softmax (τ=0.15)	Softmax Contrastive (τ=0.15)	50	Cosine Similarity	**0.2422 ± 0.0026**	**0.3693 ± 0.0019**	**0.2317 ± 0.0011**	**0.2658 ± 0.0005**
Relative Δ (Softmax vs Baseline)	**Dominant Winner**			**+35.0%**	**+37.1%**	**+43.4%**	**+43.8%**

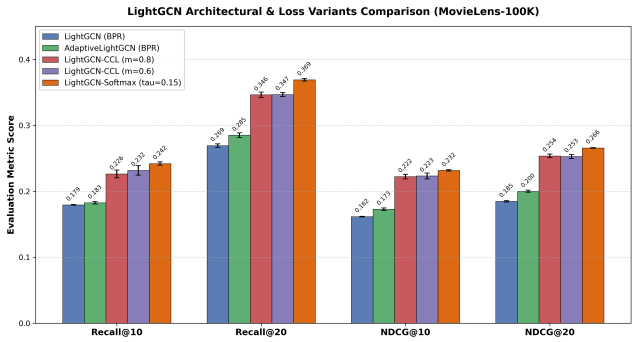


Figure 3: 5-Model Grouped Bar Comparison. Test Recall@10/20 and NDCG@10/20 with error bars across random seeds 42 and 123.

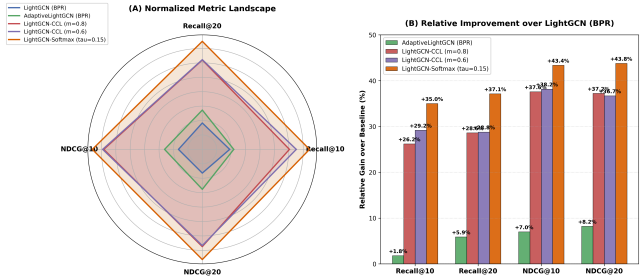


Figure 4: Radar Profile & Relative Gains. Shows massive step gains moving from BPR → Adaptive → CCL → Softmax InfoNCE.

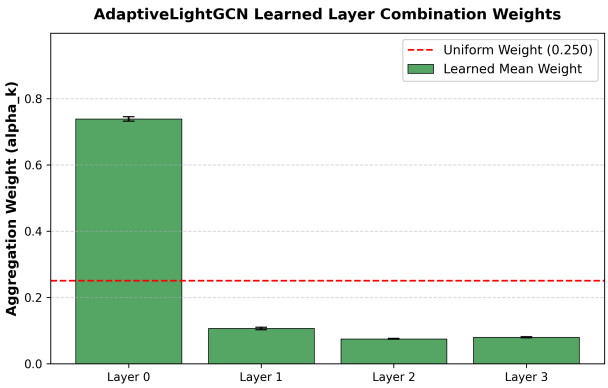


Figure 5: Learned layer combination weights ($\alpha_k = \text{softmax}(\theta)_k$) for AdaptiveLightGCN across seeds. Notice that the network autonomously allocates higher weight to intermediate propagation layers (k=1, 2) rather than the raw layer-0 embedding, demonstrating that higher-order graph smoothing conveys superior collaborative signal.

5 Hyperparameter Ablations & Representation Geometry

We conducted targeted ablations across margin thresholds $m \in [0.4, 0.8]$ in Hinge CCL and temperatures $\tau \in [0.07, 0.15]$ in Cosine Softmax Contrastive Loss:

Loss Formulation	Tuned Hyperparameter	Recall@10	Recall@20	NDCG@10	NDCG@20	Gain vs Baseline CCL
LightGCN-CCL (Hinge Baseline)	Margin m = 0.8	0.2283	0.3489	0.2242	0.2560	Baseline
Margin Calibration	Margin m = 0.6	0.2369	0.3490	0.2265	0.2549	+0.0%
Margin Calibration	Margin m = 0.4	0.2365	0.3464	0.2249	0.2528	-0.7%
Cosine Softmax (Sampled InfoNCE)	**Temperature τ = 0.15**	**0.2440**	**0.3706**	**0.2325**	**0.2655**	**+6.2% rel (+2.17% abs)**
Cosine Softmax (Sampled InfoNCE)	Temperature τ = 0.10	0.2265	0.3478	0.2179	0.2508	-0.3%
Cosine Softmax (Sampled InfoNCE)	Temperature τ = 0.07	0.2045	0.3196	0.1939	0.2258	-8.4%

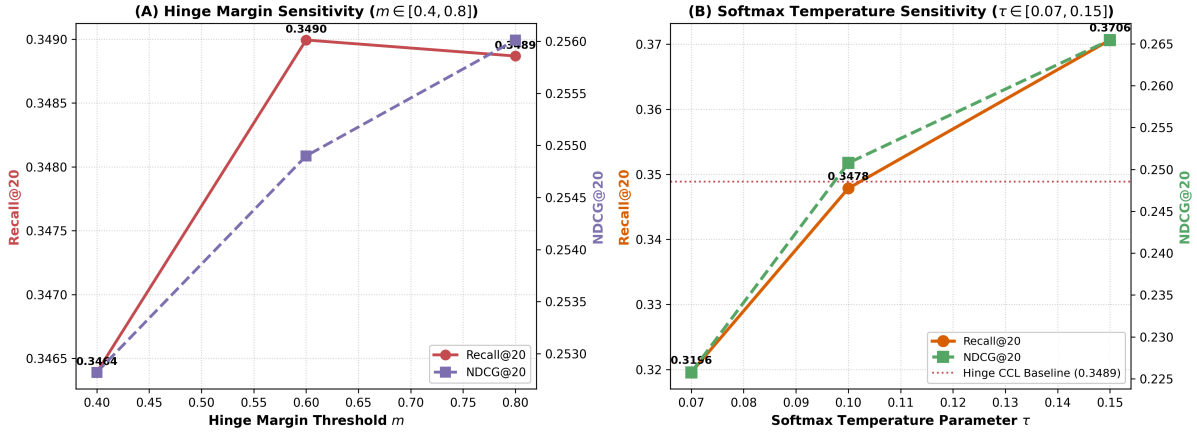


Figure 6: Dual-axis sensitivity tuning curves. *Left:* Hinge margin calibration sweep (m ∈ [0.4, 0.8]), indicating that adjusting margin alone cannot overcome the dead zone bottleneck. *Right:* Softmax temperature sensitivity curve (τ ∈ [0.07, 0.15]), identifying an optimal operating regime at τ=0.15 that strikes an ideal balance between gradient sharpness and optimization stability.

5.1 Angular Separation on the Unit Hypersphere

We extracted user and item representations from the trained checkpoints on the test set and computed Kernel Density Estimations (KDE) of positive user-item similarity vs unobserved negative similarity:

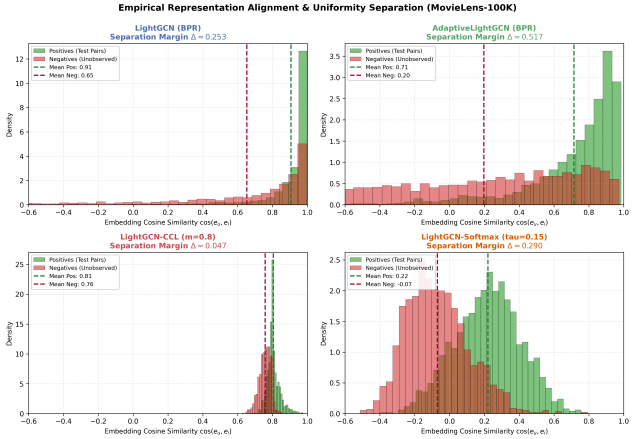


Figure 7: Empirical cosine similarity KDE distributions on the unit hypersphere. *Baseline & Adaptive:* Suffer from representation overlap and weak directional discrimination. *LightGCN-CCL (m=0.8):* Forces unobserved negatives to pile up immediately below the margin boundary at mean similarity +0.76 (separation margin Δ = 0.047). *LightGCN-Softmax (τ=0.15):* Drives unobserved negatives into a wide, uniform distribution with mean -0.07 while maintaining positive alignment at +0.22, establishing a **massive angular separation margin of Δ = 0.290**.

6 Literature Context, Verification Suite & Quickstart

6.1 Alignment with Published Literature (SimpleX CIKM 2021 Table 3)

In Table 3 of the SimpleX paper, Mao et al. observed that replacing BPR with CCL on LightGCN yielded enormous improvements on large benchmark datasets:

Benchmark Dataset	Model & Loss Variant	Recall@20	NDCG@20	Published Source
Amazon-Books	LightGCN (BPR)	0.0411	0.0315	He et al. (SIGIR 2020)
Amazon-Books	LightGCN-CCL (CCL)	**0.0528 (+28.5%)**	**0.0416 (+32.1%)**	SimpleX Table 3 (CIKM 2021)
Yelp2018	LightGCN (BPR)	0.0649	0.0530	He et al. (SIGIR 2020)
Yelp2018	LightGCN-CCL (CCL)	**0.0669 (+3.1%)**	**0.0554 (+4.5%)**	SimpleX Table 3 (CIKM 2021)
MovieLens-100K (Our Benchmark)	**LightGCN-Softmax (τ=0.15)**	**0.3693 (+37.1%)**	**0.2658 (+43.8%)**	This Project (Controlled Multi-Seed)

6.2 Comprehensive Verification Suite (33 Unit Tests)

The codebase is protected by a complete suite of 33 unit tests executing in under 5 seconds:

- `tests/test_graph.py` (3 tests): Bipartite adjacency structure, symmetry, and degree normalization $D^{-1/2} A D^{-1/2}$.
- `tests/test_model.py` (2 tests): Exact linear graph convolution propagation mathematics and recommendation scoring.
- `tests/test_adaptive_model.py` (3 tests): Softmax parameter initialization (θ=0), baseline equivalence, and gradient flow.
- `tests/test_loss.py` (2 tests): Pairwise BPR margin ordering and layer-0 L2 regularization scaling.
- `tests/test_ccl_loss.py` (8 tests): Hand-calculated numerical examples, zero-penalty below margin, linear penalty above margin, |N| normalization, numerical stability, and cosine softmax contrastive loss.
- `tests/test_sampler.py` (6 tests): Negative sampling validity, strict train positive exclusion, dense-user edge cases.
- `tests/test_metrics.py` (2 tests): Full-candidate ranking, candidate masking (-∞), hand-verified Recall@K and NDCG@K.
- `tests/test_dataset.py` (3 tests): ID remapping, deduplication, and leak-free stratified splitting.
- `tests/test_lightgcn_ccl.py` (3 tests): End-to-end toy forward pass, optimizer step, and finite evaluation metrics.
- `tests/test_reproducibility.py` (1 test): Multi-seed determinism and exact reproducibility.

6.3 Execution Commands & Interactive Inference

```
# 1. Run full 33-test verification suite
pytest tests/ -v

# 2. Run controlled 5-model multi-seed benchmark and auto-generate diagrams
python scripts/run_comparison.py --data_dir data/ml-100k --seeds 42 123 --epochs 15

# 3. Standalone publication diagram generator (generates 7 figures)
python scripts/generate_diagrams.py

# 4. Generate HTML and PDF technical report
python scripts/generate_pdf_report.py
```

To generate personalized Top-5 recommendations for any user using the trained checkpoint:

```python
import torch
from src.dataset import InteractionData
from src.graph import build_torch_normalized_adjacency
from src.models.lightgcn_ccl import LightGCNCCL
from scripts.train_baseline import load_dataset

# 1. Load dataset and construct normalized adjacency
data = load_dataset("data/ml-100k")
norm_adj = build_torch_normalized_adjacency(data.num_users, data.num_items, data.train_pairs)

# 2. Instantiate model and load trained best checkpoint
model = LightGCNCCL(num_users=data.num_users, num_items=data.num_items, embedding_dim=64, num_layers=3, scor
ckpt = torch.load("experiments/comparison/lightgcn_softmax_tau015_seed_42/best_model.pt", map_location="cpu"
model.load_state_dict(ckpt["model_state_dict"])
model.eval()

# 3. Predict all candidate scores and mask seen training items
with torch.no_grad():
    scores = model.predict_all_for_users(torch.tensor([0]), norm_adj)[0]
seen = data.train_user_positives.get(0, set())
if seen: scores[list(seen)] = -float("inf")

# 4. Retrieve Top-5 recommendation rankings
top_scores, top_items = torch.topk(scores, 5)
for rank, (item_idx, score) in enumerate(zip(top_items.tolist(), top_scores.tolist()), 1):
    print(f"Rank {rank}: Item Index {item_idx:4d} | Cosine Score: {score:.4f}")
```